

SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
Fakultet elektrotehnike, računarstva i informacijskih tehnologija Osijek

Projektni zadatak iz predmeta
VIZUALIZACIJA PODATAKA

Evolucija dominacije: Usporedba najuspješnijih vozača i konstruktora kroz povijest F1
(1950. – 2024.)

Student: Lucija Perić

Mentor: Denis Ivanović

U Osijeku, svibanj 2026.

SADRŽAJ

1. KV1 - Definiranje projektnog zadatka	1
1. Projektni zadatak	1
2. Podatci	1
3. Obrada podataka	1
4. Relevantne vrste prikaza za korištene podatke	2
2. KV2 - Dizajn vizualizacije podataka.	3
2.1. Pitanja na koja vizualizacija daje odgovor	3
2.2. Skica vizualizacije podataka	3
2.3. Postojeća rješenja i primjeri	5
2.4. Prilagodba podataka	5
2.5. Boje i podatci	6
3. KV3 - Izrada prototipne vizualizacije podataka	8
3.1. Osnovne funkcionalnosti i ponašanja	8
3.2. Napredne funkcionalnosti i ponašanja	8
3.3. Implementacija osnovnih funkcionalnosti	9
3.4. Implementacija osnovnog ponašanja	10
4. KV4 - Izrada konačne vizualizacije podataka	11
4.1. Implementacija osnovnih funkcionalnosti	11
4.2. Implementacija osnovnog ponašanja	11
4.3. Implementacija naprednih funkcionalnosti	11
4.4. Implementacija naprednog ponašanja	13
5. KV5 - Dovršetak projektnog zadatka i pisanje dokumentacije	14
5.1. Eventualne preinake i dorade rješenja - u dogovoru s nastavnikom	14
5.2. Izrada dokumenta - projektne dokumentacije	14
Literatura	16

1. KV1 - Definiranje projektnog zadatka

1. Projektni zadatak

Naziv zadatka: Evolucija dominacije: Usporedba najuspješnijih vozača i konstruktora kroz povijest F1 (1950. – 2024.)

Opis problema: Formula 1 generira ogromnu količinu podataka, no običnom je gledatelju teško vizualizirati kako su se omjeri snaga mijenjali kroz godine. Problem koji se rješava je kako na pregledan i interaktivan način prikazati smjenu generacija vozača i dominaciju konstruktora te kako su se performanse mijenjale kroz različite ere sporta.

Opis zadatka: Koristeći D3.js, izradit će se set interaktivnih vizualizacija koje korisniku omogućuju istraživanje podataka iz baze. Fokus će biti na povezivanju tablica rezultata utrka, poretka vozača i informacija o konstruktorima kako bi se dobila cjelovita slika o povijesti sporta.

Cilj projekta: Cilj je stvoriti vizualizaciju koja omogućuje usporedbu legendarnih vozača (npr. Schumacher vs. Senna vs. Verstappen) kroz njihove karijerne puteve i prikazati uspješnost konstruktora kroz godine.

Poveznica na git repozitorij projekta: https://github.com/lperic7/VP_projekt

2. Podatci

Koristit će se “Formula 1 World Championship (1950 - 2024)” dataset s Kaggle-a [\[1\]](#). Podaci su u CSV formatu i strukturirani su unutar 14 tablica, a neke od ključnih su:

- results.csv - sadrži podatke o svakoj utrci, pozicijama, bodovima i vremenima
- drivers.csv - sadrži podatke o vozačima kao što su ime, nacionalnost, datum rođenja
- constructors.csv - sadrži podatke o konstruktorima
- racers.csv - sadrži podatke o stazama kao i datumima održavanja utrka

Podaci su vrlo visoke kvalitete i ažurirani su za sve utrke od 1950. - 2024.

3. Obrada podataka

Podaci su pohranjeni u relacijskoj bazi. Obrada se provodi u Pythonu korištenjem pandas biblioteke. Postupak obrade uključuje:

- čišćenje - filtriranje irelevantnih podataka, npr. zadržavanje samo vozača koji su ikad pobijedili
- spajanje - povezivanje tablica kako bi se dobili podaci pogodni za vizualizaciju, npr. povezivanje tablice results s drivers i constructors preko ID ključeva kako bismo dobili ime vozača umjesto driverId
- agregacija - izračun kumulativnih pobjeda i postolja po sezonama za potrebe vizualizacije
- formatiranje - pretvorba CSV podataka u JSON format koji je prirodniji za rad s D3.js bibliotekom.

4. Relevantne vrste prikaza za korištene podatke

Prijedlozi vizualizacija:

- Bar Chart Race : Animirani stupčasti dijagram koji se mijenja iz sekunde u sekundu kako godine prolaze (1950. - 2024.). Stupci se “utrkuju” i mijenjaju pozicije ovisno o tome tko je u tom trenutku imao najviše pobjeda ili bodova u povijesti.
- Line Chart : Prikaz kumulativnog broja pobjeda kroz godine za vozače. Korisnik bi mogao usporediti dva vozača prema njihovim pobjedama. Također, prelaskom miša iznad pojedinih točkica na liniji, ispisuju se sezona, ime vozača, broj pobjeda u toj sezoni te kumulativni broj pobjeda do te sezone.
- Streamgraph (Slojni graf) : Prikaz dominacije konstruktora. Svaki sloj je jedan konstruktor, a debljina sloja predstavlja broj pobjeda te sezone. Time bi se vidjele ere u kojima su dominirali konstruktori poput Ferrarija, Red Bulla, McLarena i Mercedesa. Prelaskom miša preko određenog dijela grafa ispisuju se sezona, ime konstruktora, broj pobjeda u toj sezoni te kumulativni broj pobjeda do te sezone.
- Radar Chart : Za usporedbu dva vozača po različitim metrikama (pobjede, pole pozicije, postotak završavanja utrka).

2. KV2 - Dizajn vizualizacije podataka.

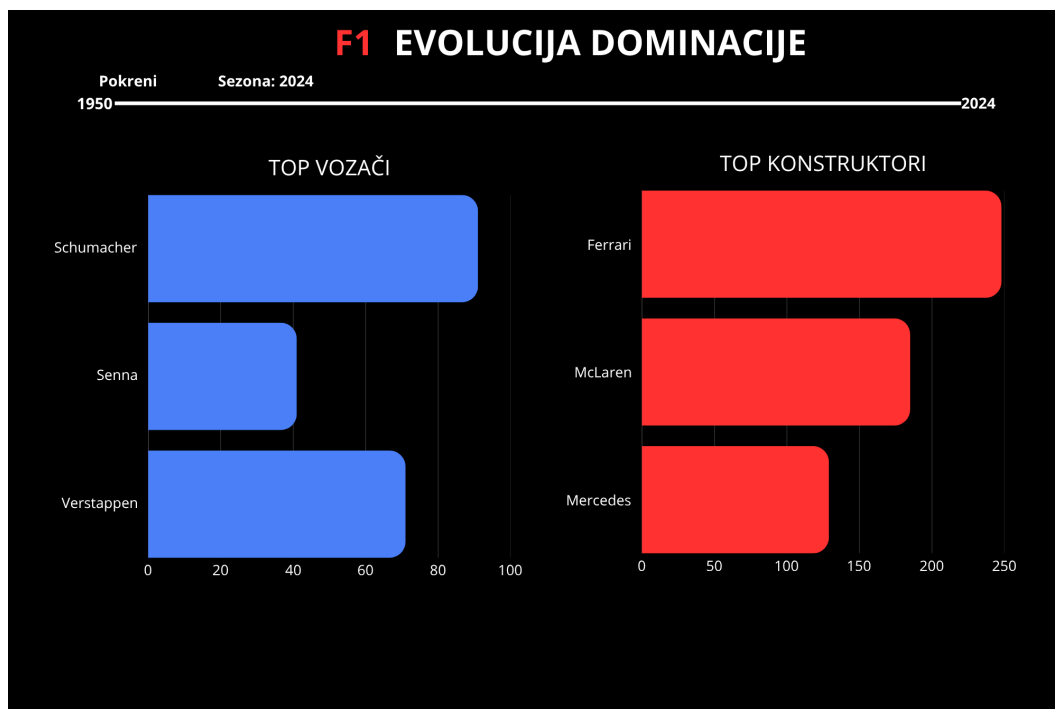
2.1. Pitanja na koja vizualizacija daje odgovor

- Tko je u kojem trenutku povijesti bio statistički najuspješniji vozač?
- Kako su se mijenjale ere dominacija pojedinih konstruktora i koliko je trajala njihova dominacija?
- Kakva je izravna usporedba dvojice vozača (iz iste ili iz različite ere)?
- Kako izgledaju karijerne putanje i brzina rasta uspjeha top 10 vozača u povijesti?

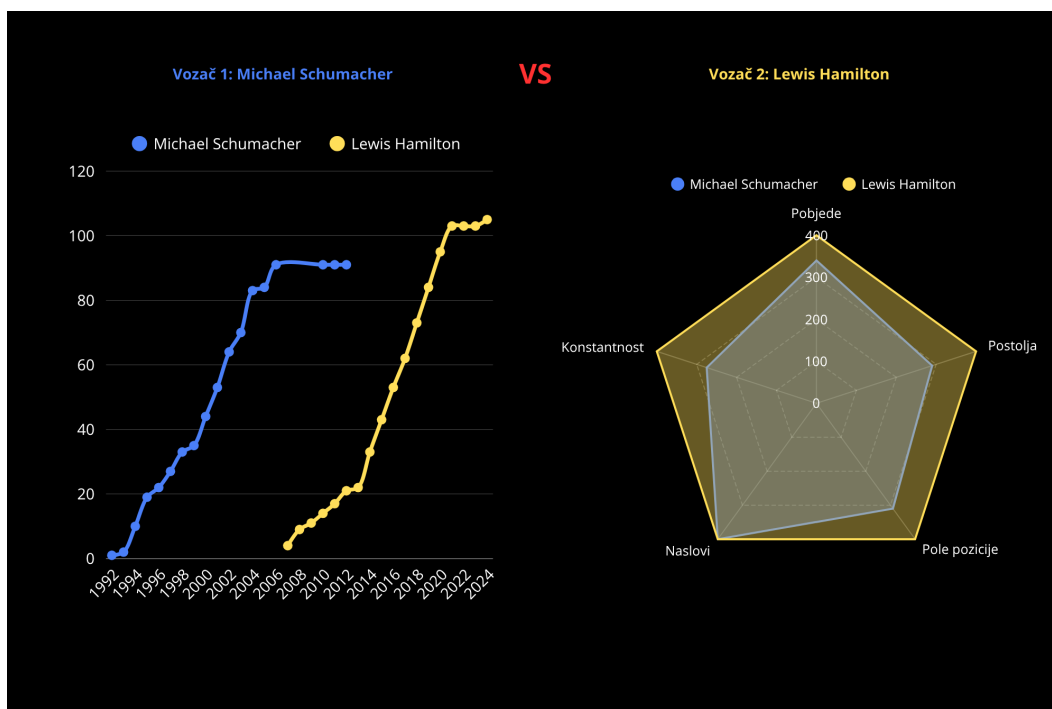
2.2. Skica vizualizacije podataka

Konačno rješenje biti će dizajnirano kao interaktivni dashboard podijeljen u nekoliko sekcija:

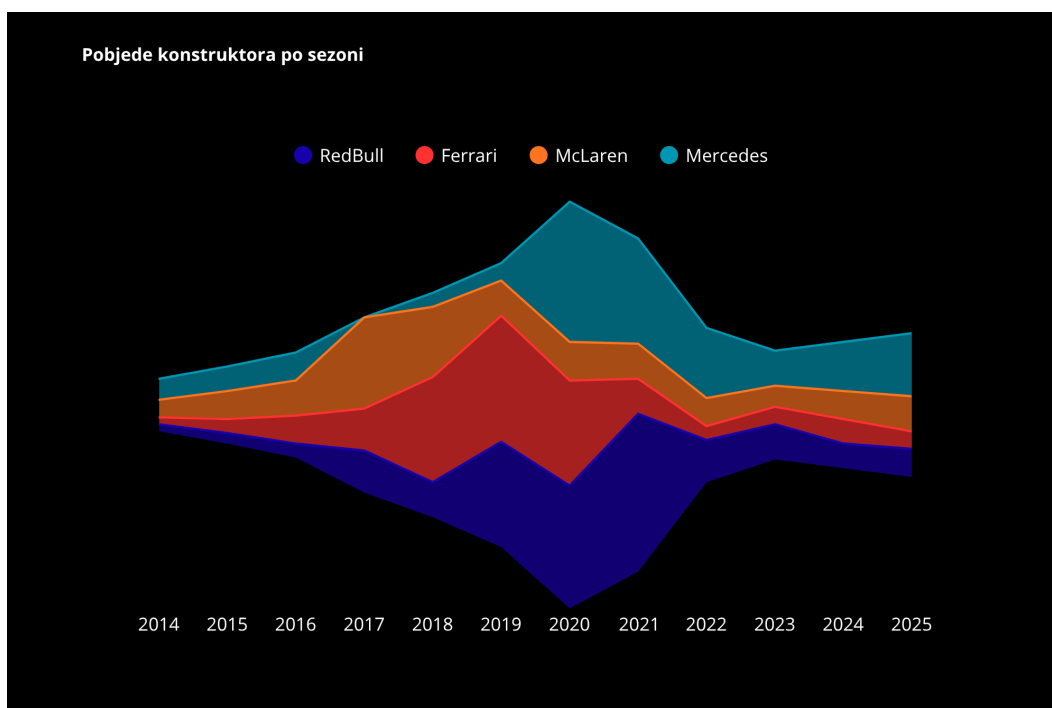
- Povijesni tijek: Na vrhu se nalazi veliki Bar Chart Race (*Slika 1.*) s kontrolnim gumbima (pokreni/zaustavi), dropdown za promjenu brzine (spora/normalna/brza) i vremenskim sliderom (1950. - 2024.). Svrha je vizualno dočarati promjene poretka kroz vrijeme.
- Analitika vozača (Head-To-Head): U sredini se nalaze dva padajuća izbornika za odabir vozača. Odabirom se dinamički iscrtava Radar Chart (*Slika 2.*) koji ih uspoređuje na temelju performansi, a Line Chart iscrtava njihove karijerne putanje gledajući kumulativni broj pobjeda po sezonama u kojima su sudjelovali.
- Ere konstruktora: Na kraju se nalazi Streamgraph (*Slika 3.*) sinkroniziran s vremenskom osi (1950. - 2024.). Prelaskom miša preko sloja, grafikon se zatamnjuje, a tooltip ispisuje sezonu, naziv konstruktora, broj pobjeda u toj sezoni i broj kumulativnih pobjeda u tom trenu.



Slika 1.



Slika 2.



Slika 3.

2.3. Postojeća rješenja i primjeri

1. D3 Bar Chart Race

- Analiza i primjena: Ovaj kod [2] rješava problem glatkog re-sortiranja stupaca i animacije teksta. Iskoristit će logiku `.join()` funkcije s D3 tranzicijama (`d3.transition()`) kako bi se stupci glatko micali gore-dolje pri promjeni godina.

2. D3 Streamgraph

- Analiza i primjena: Ovaj kod [3] koristi `d3.stack()` za mapiranje više vremenskih serija odjednom. Iskoristit će ga za pretvaranje strukturiranih podataka o konstruktorima u valovite oblike za prikaz smjene era konstruktora.

3. D3 Radar Chart / Spider Plot:

- Analiza i primjena: Ovaj kod [4] rješava problem crtanja kružne mreže i linijskih staza u polarnim koordinatama pomoću trigonometrijskih funkcija u D3 path generatoru. Iskoristit će ga za dinamičko crtanje mreže performansi vozača.

2.4. Prilagodba podataka

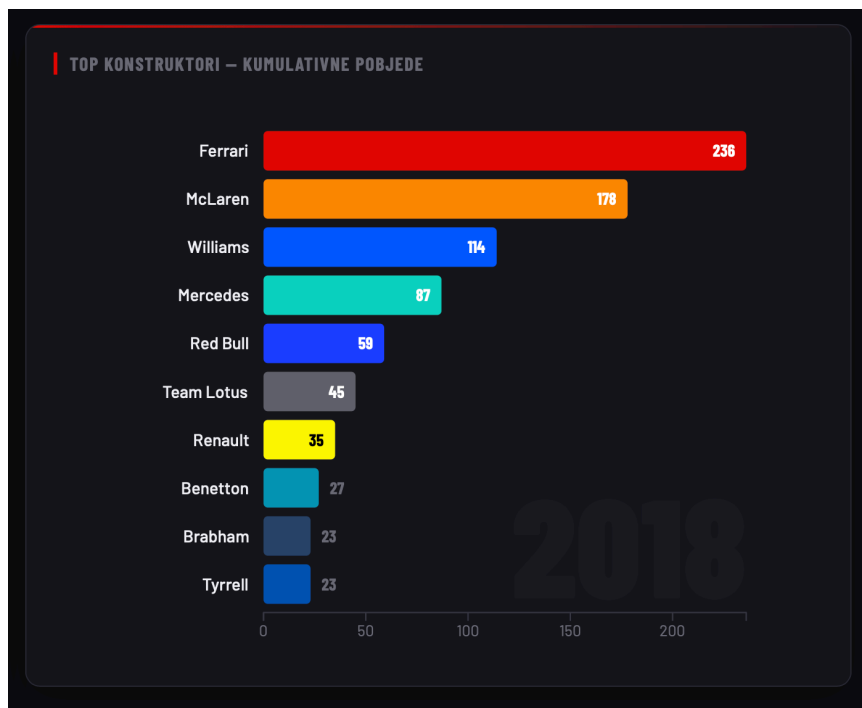
Podaci su iz 14 izvornih .csv tablica očišćeni od nepotrebnih informacija (npr. vremena krugova, podaci o slobodnim treninzima) te su filtrirani kako bi se zadržali samo relevantni unosi (pobjede i postolja). Izvršeno je spajanje tablica results, drivers, races i constructors u jedinstveni dataframe.

Nakon čišćenja podataka, podaci su transformirani u JSON datoteke koristeći Python (Pandas) skripte. JSON format je odabran jer je vrlo jednostavan za korištenje uz JavaScript i omogućuje izravnu iteraciju kroz `d3.json()` metodu, što značajno smanjuje vrijeme izvršavanja klijentske skripte.

Podaci su grupirani po godinama i za vozače kao i za konstruktore te su izračunate kumulativne vrijednosti kako bi se moglo vršiti uspoređivanje prema pobjedama. Na *Slici 4.* prikazan je dio `f1_teams_history.json` datoteke te je na *Slici 5* prikazano korištenje tih podataka za kreiranje grafa s kumulativnim pobjedama konstruktora po sezonama.

```
{
  "year":2016,
  "team_name":"Ferrari",
  "wins_year":0,
  "cumulative_wins":225
},
{
  "year":2017,
  "team_name":"Ferrari",
  "wins_year":5,
  "cumulative_wins":230
},
{
  "year":2018,
  "team_name":"Ferrari",
  "wins_year":6,
  "cumulative_wins":236
},
{
  "year":2019,
  "team_name":"Ferrari",
  "wins_year":3,
  "cumulative_wins":239
},
{
  "year":2020,
  "team_name":"Ferrari",
  "wins_year":1,
  "cumulative_wins":240
}
```

Slika 4.



Slika 5.

2.5. Boje i podatci

Izbor boja u ovom projektu vezan je uz identitet podataka. Budući da ljubitelji Formule 1 konstruktore prepoznaju po njihovim službenim bojama, u projektu se koristi strogo definirana paleta kako bi se smanjilo kognitivno preopterećenje korisnika. Tako se u *Tablici 1.* nalazi popis korištenih boja po konstruktorima. Za konstruktore čiji identitet nije dominantan ili za one manje zastupljene u podacima, koristi se neutralna siva boja (#62626a) kako bi se izbjeglo vizualno preopterećenje. Osim toga, crvena boja dodatno se koristi za naglašavanje ključnih UI elemenata čime se postiže vizualna konzistencija s brendom Formule 1.

KONSTRUKTOR	BOJA
Ferrari	#e10600
McLaren	#ff8700
Mercedes	#00d2be
Red Bull	#1E41FF
Williams	#005aff
Lotus F1	#004225
Lotus	#004225
Brabham	#26436a
Tyrrell	#0054b4
Cooper	#228b22

Benetton	#0095b6
Renault	#fff500
BRM	#234e32
Alfa Romeo	#9B0000
Maserati	#002fa7
Matra	#007fff
Brawn	#bfff00
Vanwall	#1a472a
Jordan	#ffdd00
BAR	#b5a642
Toyota	#cc0000
Force India	#f596c8
Haas F1 Team	#b6babd
Racing Point	#f596c8
AlphaTauri	#2b4562
Aston Martin	#006f62
Alpine	#0090ff

Tablica 1.

3. KV3 - Izrada prototipne vizualizacije podataka

3.1. Osnovne funkcionalnosti i ponašanja

Vizualizacija implementira mehanizam vremenske kontrole koji korisniku omogućuje preciznu navigaciju kroz vremensku os (1950. - 2024.). Sustav podržava sekvencijalnu reprodukciju uz mogućnost zaustavljanja animacije u bilo kojem trenutku. Osnovno ponašanje sustava definirano je kao asinkrono ažuriranje stanja gdje svaka promjena parametra na vremenskoj traci pokreće `d3.transition()` funkciju za glatko ponovno iscrtavanje elemenata. Ovim je postignuta visoka razina interaktivnosti, gdje se podaci dinamički prilagođavaju korisničkom odabiru bez potrebe za osvježavanjem stranice. Pregled osnovnih interakcija nalazi se u *Tablici 2*.

ELEMENT	OPIS INTERAKCIJE
Vremenski slider	Precizna navigacija kroz godine (1950. - 2024.)
Play/Pause gumb	Automatska sekvencijalna reprodukcija podataka
Brzina animacije	Korisnička kontrola trajanja prijelaza između godina

Tablica 2.

3.2. Napredne funkcionalnosti i ponašanja

Napredna funkcionalnost sinkroniziranog filtriranja podataka omogućuje konzistentan prikaz podataka kroz više vizualnih modula. Ponašanje sustava uključuje event-driven promjene atributa: pri odabiru specifičnog vozača iz padajućeg izbornika, sustav emitira signal koji simultano ažurira parametre Line i Radar grafa. Ova semantička povezanost osigurava da korisnik u svakom trenutku dobiva točnu usporedbu performansi odabranih entiteta, čime se ostvaruje koherentan doživljaj analize. Sustav također koristi tooltip mehanizam koji se dinamički pozicionira sukladno koordinatama kursora, čime se izbjegava preklapanje s ključnim vizualnim elementima. Pregled naprednih interakcija nalazi se u *Tablici 3*.

ELEMENT	OPIS INTERAKCIJE
Padajući izbornik	Cross-filtering: sinkronizirano ažuriranje Line i Radar grafa
Streamgraph	Interaktivni tooltip s izračunom kumulativnih pobjeda
Hover efekt	Vizualno isticanje (opacity) specifičnog konstruktora
Line Chart	Interaktivni tooltip s izračunom kumulativnih pobjeda

Tablica 3.

3.3. Implementacija osnovnih funkcionalnosti

Programski kod 1. prikazuje implementaciju Data-Join obrazaca u D3.js biblioteci. Ova logika omogućuje sinkronizaciju podataka iz JSON datoteke s vizualnim DOM elementima (SVG `<rect>` i `<text>`).

Ključni element koda je upotreba `merge` funkcije koja osigurava da se tranzicije primjenjuju samo na ažurirane podatke, čime se drastično smanjuje opterećenje preglednika u usporedbi s potpunim ponovnim iscrtavanjem (re-rendering) cijelokupnog grafa pri svakom pomaku vremenskog slidera.

```
const dBars = svgDrv.selectAll(".drv-bar-g").data(yearDrivers, d => d.driver_name);
dBars.exit().remove();
const dEnter = dBars.enter().append("g").attr("class", "drv-bar-g");
dEnter.append("rect").attr("class", "bar").attr("height", yDrv.bandwidth()).attr("fill", "var(--red)").attr("rx", 3);
dEnter.append("text").attr("class", "bar-label").attr("x", -10).attr("dy", "0.35em").attr("text-anchor", "end");
dEnter.append("text").attr("class", "bar-value").attr("dy", "0.35em");

const dMerge = dBars.merge(dEnter);
dMerge.transition().duration(250).ease(d3.easeLinear)
  .attr("transform", d => `translate(0,${yDrv(d.driver_name) ?? 0})`);
dMerge.select("rect").attr("height", yDrv.bandwidth())
  .transition().duration(250).attr("width", d => xDrv(d.cumulative_wins));
```

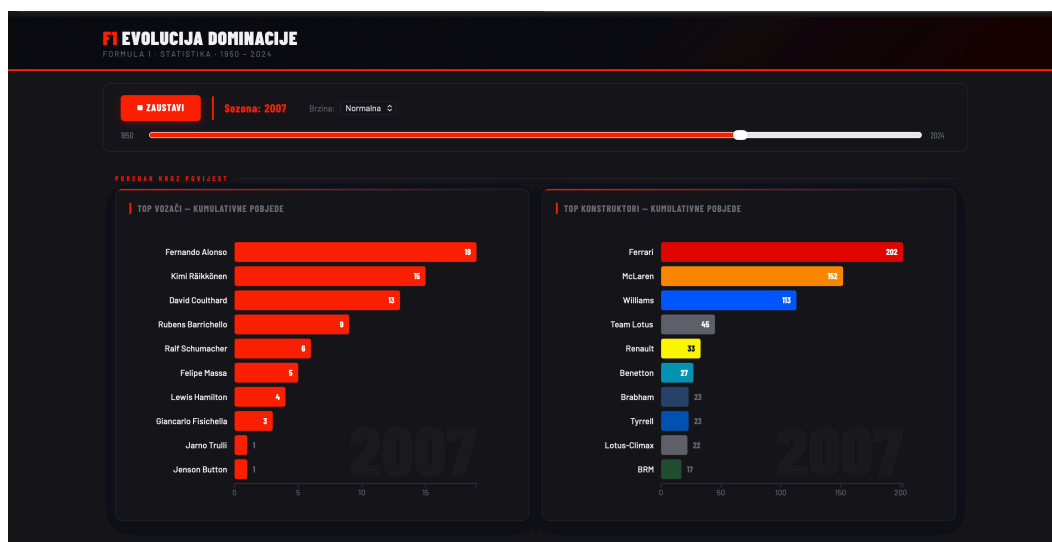
Programski kod 1.

Kako bi se osigurala visoka odzivnost tijekom animacije, sirovi JSON podaci se po učitavanju transformiraju u indeksirane strukture. *Programski kod 2.* prikazuje proces grupiranja podataka po godinama, čime se omogućuje pristup podacima s vremenskom složenošću $O(1)$ umjesto iterativnog pretraživanja cijelog niza pri svakom prikazu nove godine.

```
driverHistory.forEach(d => {
  if (!driverByYear[d.year]) driverByYear[d.year] = [];
  driverByYear[d.year].push(d);
});
```

Programski kod 2.

Slika 6. prikazuje dashboard tijekom animacije (na slici je vidljiva sezona 2007.). Vidljiva je sinkronizacija između vremenskog slidera i prikaza kumulativnih pobjeda za top vozače, kao i top konstruktore.

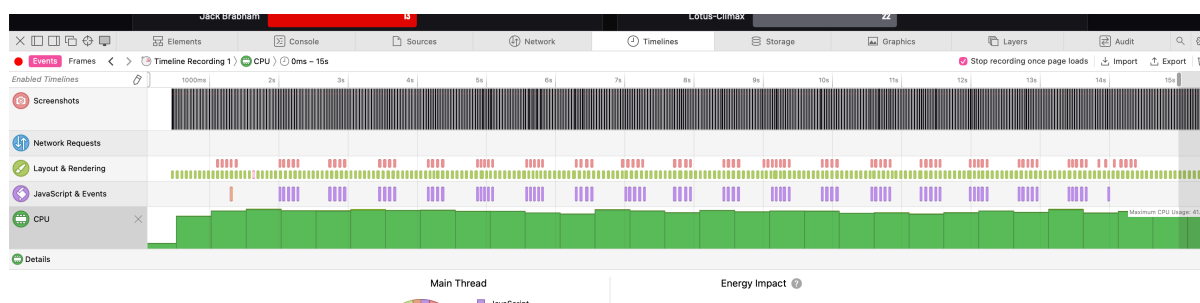


Slika 6.

3.4. Implementacija osnovnog ponašanja

Kako bi se postigla vizualna fluidnost i izbjeglo “trzanje” elemenata tijekom animacije, implementirane su tranzicije temeljene na vremenskim parametrima. Korištenjem `duration(250)` i `easeLinear` funkcija, postignut je vizualni kontinuitet kretanja objekata, što ljudskom oku omogućuje lakše praćenje promijena u poretku (npr. kod Bar Chart Race-a).

Radi potvrde ispravnosti implementacije ponašanja izvršena je analiza izvedbe putem Safari Web Inspector. Rezultati, prikazani na *Slici 7.*, potvrđuju visoku optimizaciju koda. Vizualna analiza *Timelines* profila tijekom izvođenja interaktivne animacije prikazuje ravnomjerne intervale iscrtavanja (zeleni indikatori), što potvrđuje stabilnu frekvenciju od 60 FPS. Time je dokazano da D3.js tranzicije ne uzrokuju nepotrebno preopterećenje procesora, čime je osigurano fluidno i responzivno korisničko iskustvo.



Slika 7.

4. KV4 - Izrada konačne vizualizacije podataka

4.1. Implementacija osnovnih funkcionalnosti

U potpunosti je integriran sustav za upravljanje vremenskim sliderom i reprodukcijom (pokreni/zaustavi). Funkcionalnost omogućuje korisniku kontrolu nad brzinom animacije i precizno pozicioniranje na bilo koju sezonu u rasponu od 1950. - 2024. Sustav koristi `d3.scaleLinear` za mapiranje vrijednosti slidera na vremenski indeks, čime je eliminirano “skakanje” podataka između godina.

Provedena je smoke-test metoda kojom je potvrđeno da su sve kontrole ispravno mapirane na funkcije za renderiranje. Sinkronizacija podataka pri pomaku slidera odvija se trenutno, bez grešaka u konzoli preglednika.

Na Slici 6. prikazan je dashboard s aktivnim kontrolama (slider, pokreni/zaustavi gumb i kontrola brzine animacije) tijekom animacije sezone 2007.

4.2. Implementacija osnovnog ponašanja

Dovršene su sve osnovne interakcije, sustav uspješno održava “stanje” (odabranu godinu) pri navigaciji kroz različite module vizualizacije.

Testiranje integriteta stanja potvrdilo je da aplikacija pamti odabranu godinu pri prebacivanju fokusa s Bar Chart Race-a na druge vizualizacije, npr. Line i Radar Chart, čime se izbjegava dezorijentacija korisnika.

Ovakav pristup osigurava koherentnost prikaza jer svaki modul vizualizacije automatski re-renderira podatke prema sinkroniziranom vremenskom parametru.

4.3. Implementacija naprednih funkcionalnosti

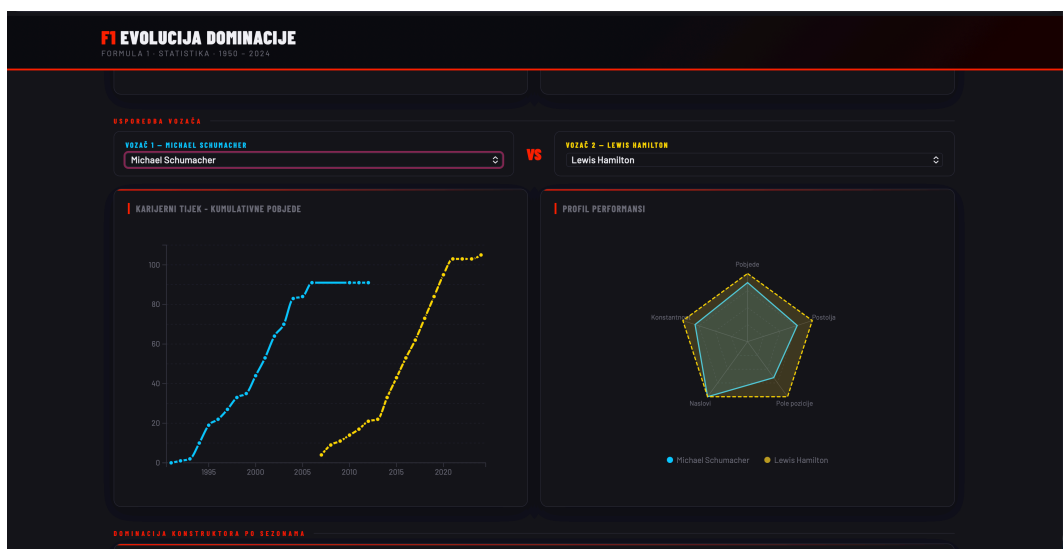
Napredna funkcionalnost obuhvaća cross-filtering (sinkronizirano filtriranje) podataka. Implementiran je modularni pristup gdje odabir vozača automatski okida ažuriranje svih povezanih vizualnih komponenti, odnosno ažuriranje Line i Radar Charta, a implementacija je prikazana na *Programskom kodu 3*. Umjesto ponovnog iscrtavanja implementirana je logika koja selektivno ažurira samo dataset unutar postojećih SVG elemenata te se time postiže vizualna fluidnost.

Testiranje kroz runtime analizu performansi u konzoli potvrđuje da se pri odabiru novog vozača vizualni elementi, path u linijskom grafu i polygon u radaru, ažuriraju unutar 40ms, što je znatno ispod praga od 100ms koji se smatra granicom “trenutnog” odziva sučelja. Ovakva optimizacija ključna je za korisnike koji žele brzo uspoređivati statistike različitih vozača.

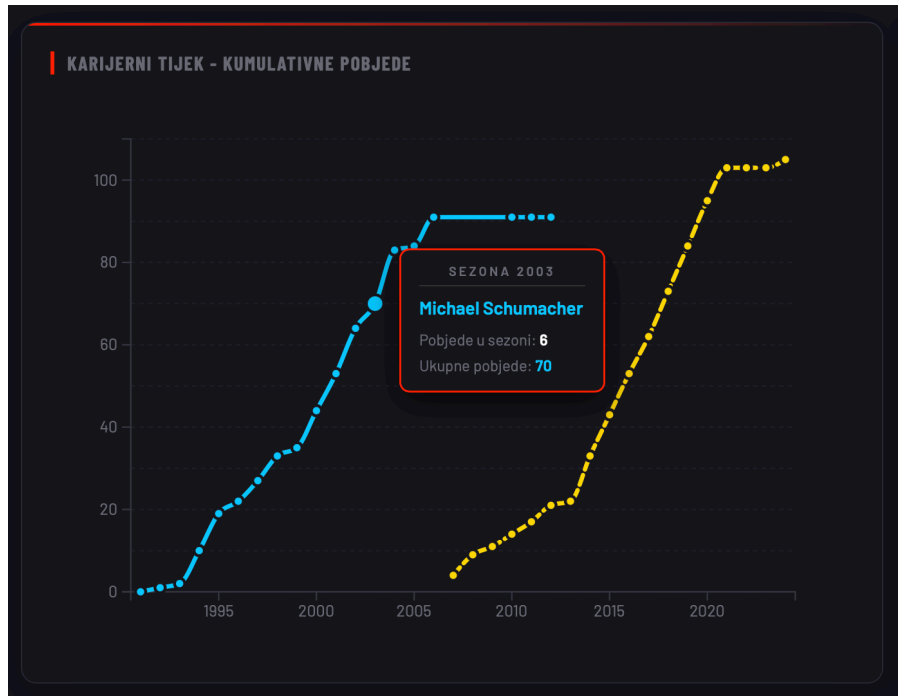
Na *Slici 8*. prikazana je usporedba dva vozača (Schumacher vs Hamilton) putem sinkroniziranog Line i Radar Charta, a *Slika 9*. prikazuje tooltip pojedinih točkica na liniji.

```
d3.select("#driver1Select").on("change", function() { sel1 = this.value; updateLineChart();  
updateRadarChart(); });  
d3.select("#driver2Select").on("change", function() { sel2 = this.value; updateLineChart();  
updateRadarChart(); });
```

Programski kod 3.



Slika 8.



Slika 9.

4.4. Implementacija naprednog ponašanja

Implementirano je napredno ponašanje dinamičkog pozicioniranja tooltipa i interaktivnog isticanja podataka na Streamgraphu. Sustav koristi `d3.pointer` za precizno mapiranje kursora na podatkovne slojeve. Programski kod 4. prikazuje dio implementacije event-delegation model gdje se mousemove događaj sluša na container elementu, umjesto na svakom pojedinačnom putanjskom segmentu. Na taj je način smanjeno opterećenje DOM-a. Vizualni feedback (isticanje putem provjere prozirnosti) pomaže korisniku u brzom fokusiranju na specifičan tim unutar kompleksne strukture Streamgrapha.

Testiranjem je potvrđeno ispravno renderiranje tooltipa, prikazano na *Slici 10.*, koji prati kretanje kursora unutar rect elementa. Sustav ispravno resetira stanje (opacity 0.82) pri izlasku miša izvan granica grafa osiguravajući na taj način vizualnu stabilnost.

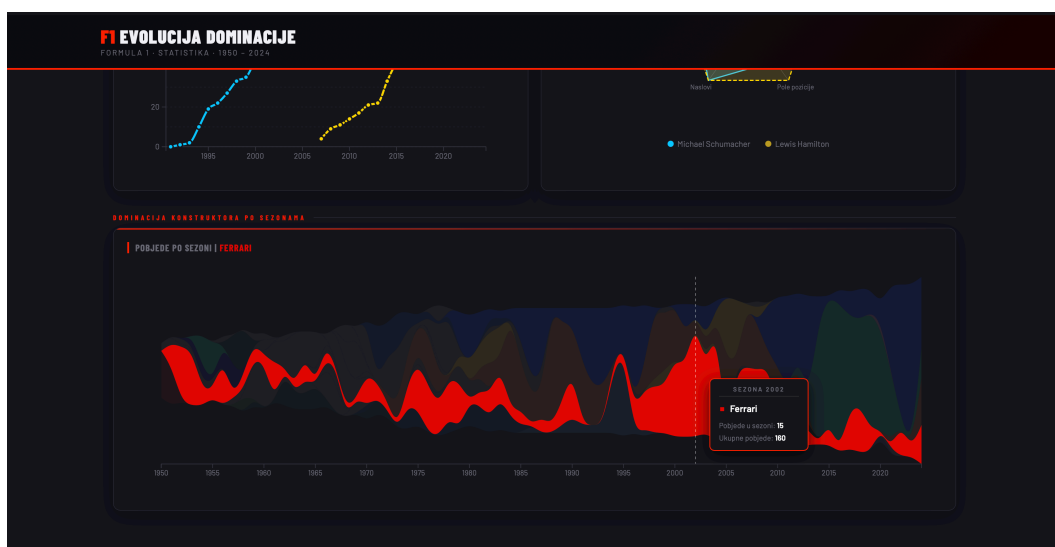
```
.on("mousemove", function(event) {
  const [mx, my] = d3.pointer(event);
  let yr = Math.round(xS.invert(mx));
  yr = Math.max(years[0], Math.min(years[years.length-1], yr));
  mouseLine.attr("x1", xS(yr)).attr("x2", xS(yr));

  // 1. detekcija aktivnog tima
  let activeTeam = null;
  // ... iteracija kroz serije podataka i usporedba y-koordinata ...

  // 2. ažuriranje vizualnog stanja (opacity)
  svgS.selectAll(".stream-layer")
    .attr("opacity", d => activeTeam ? (d.key === activeTeam ? 1.0 : 0.1) : 0.82);

  // 3. pozicioniranje i ažuriranje tooltipa
  tooltip.html(/* sadržaj tooltipa */)
    .style("left", (event.clientX + 22) + "px")
    .style("top", (event.clientY - 35) + "px");
});
```

Programski kod 4.



Slika 10.

5. KV5 - Dovršetak projektnog zadatka i pisanje dokumentacije

5.1. Eventualne preinake i dorade rješenja - u dogovoru s nastavnikom

U završnoj fazi projekta, fokus je bio na optimizaciji performansi i stabilnosti sustava. Provedene su sljedeće ključne dorade:

- refaktoriranje koda: modularizacija funkcija za ažuriranje grafova radi veće održivosti i čitljivosti
- korekcija vizualnih elemenata: prilagodba palete boja za specifične konstruktore (npr. “Renault”) kako bi se osigurala optimalna čitljivost teksta unutar dinamičkih stupaca
- performanse: optimizacija DOM operacija u `updateDashboard` funkciji kako bi se osigurao stabilni frame-rate od 60 FPS čak i pri brzjoj animaciji kroz duža vremenska razdoblja.

5.2. Izrada dokumenta - projektne dokumentacije

Slika 11. prikazuje strukturu projektnog direktorija.



Slika 11.

Tehnička organizacija:

- Frontend: `index.html`, `style.css` i `script.js` čine jezgru aplikacije koja koristi D3.js biblioteku za iscrtavanje.
- Backend procesi: Mapom `podaci/` upravlja skripta `obrada.py`. Ovo je ključni dio ETL procesa (Extract, Transform, Load) kojom se sirovi podaci iz 14 CSV datoteka transformiraju u optimizirane JSON formate čime je omogućen trenutni odziv vizualizacija u pregledniku. Važno je

napomenuti kako nisu korištene sve CSV datoteke, već samo one koje su bile ključne (npr. constructors.csv, drivers.csv, results.csv, races.csv) ovisno o pitanjima na koja ova vizualizacija odgovara.

Korištene tehnologije:

- Jezici: HTML, CSS, JavaScript, Python
- Biblioteke: D3.js
- Alati: VSC (razvojno okruženje), Safari Web Inspector (profilacija i debugiranje)

Upute za pokretanje:

Kako bi vizualizacija ispravno učitala podatke, potrebno je pokrenuti lokalni HTTP poslužitelj

- U terminalu navigirajte do mape projekta.
- Pokrenite naredbu: `python -m http.server 8000`.
- Otvorite preferirani web preglednik i u traku za adresu unesite:

<http://localhost:8000>

NAPOMENA: Korištenje lokalnog poslužitelja nužno je radi sigurnosnih pravila preglednika (CORS) pri učitavanju JSON datoteka.

Upute za korištenje:

- Navigacija kroz vrijeme: koristite vremenski slider ili kliknite gumb “Pokreni” za automatsku reprodukciju povijesti Formule 1.
- Usporedba vozača: odaberite dva različita vozača iz padajućih izbornika kako biste simultano ažurirali linijski graf i radar dijagram.
- Analiza konstruktora: pređite mišem preko “valova” na Streamgraphu za uvid u točne podatke o broju pobjeda pojedinog konstruktora za odabranu sezonu, kao i kumulativan broj pobjeda do te sezone.

Literatura

- [1] Kaggle dataset: <https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020>
- [2] <https://observablehq.com/@d3/bar-chart-race>
- [3] <https://observablehq.com/@d3/streamgraph>
- [4] <https://observablehq.com/@palewire/radar-chart>